

Absolutely. Since the learners have **basic Java hands-on but essentially zero programming literacy**, I would teach JavaScript from first principles, but make **every example insurance-related**. This will also make the transition to Playwright much easier later.

A good approach is to use a fictional insurance application such as “**ABC Insurance**” throughout Days 4–5.

Day 4 — JavaScript Fundamentals for Testers – I

Insurance-domain teaching approach

1. Why Testers Need Programming

Start with a simple insurance scenario:

Requirement:

If the applicant's age is 18 or above, the application can proceed. Otherwise, display "Applicant is not eligible."

Explain that a manual tester might check this manually, but an automation tester needs to express the rule in code.

JavaScript

```
let age = 25;

if (age >= 18) {
  console.log("Eligible for insurance");
} else {
  console.log("Not eligible for insurance");
}
```

Then connect it directly to automation:

Manual test thinking

Enter age → click Submit → verify eligibility message

Programming thinking

Store age → apply condition → compare result → report outcome

This is the mindset you want them to develop.

2. What is JavaScript?

Explain simply:

JavaScript is a programming language that allows us to give instructions to a computer.

For testers:

JavaScript allows us to create test conditions, manipulate test data, validate results, and eventually automate browsers using Playwright.

Insurance example:

JavaScript

```
let policyType = "Health";  
  
console.log(policyType);
```

Output:

Health

Explain:

- `let` → creates a variable
- `policyType` → variable name
- `"Health"` → value
- `console.log()` → displays the value

3. Development Environment Setup

For the training, I recommend:

- Node.js
- Visual Studio Code
- VS Code Terminal
- Chrome
- Developer Console

Have students verify:

Bash

```
node --version
```

and:

Bash

```
npm --version
```

Then create:

```
insurance-js-training  
  day4  
    insurance.js
```

Run:

Bash

```
node insurance.js
```

4. First JavaScript Program

Use an insurance example immediately:

JavaScript

```
console.log("ABC Insurance");  
console.log("Policy Application");  
console.log("Applicant: Bala");
```

Then modify:

JavaScript

```
console.log("Starting insurance test");  
console.log("Entering applicant details");  
console.log("Submitting application");  
console.log("Validating result");
```

This is a nice opportunity to explain that **JavaScript executes instructions from top to bottom**.

5. Statements and Syntax

Use small examples.

JavaScript

```
let applicantName = "Ravi";  
let age = 35;  
  
console.log(applicantName);  
console.log(age);
```

Introduce:

- statement
- semicolon
- quotes
- parentheses
- braces
- case sensitivity

Then deliberately show errors:

JavaScript

```
let applicantName = "Ravi"  
console.log(applicantName);
```

Explain that semicolons are optional in many JavaScript situations, but beginners should initially use them consistently.

6. Variables

Use insurance data.

JavaScript

```
let applicantName = "Priya";  
let age = 32;  
let premium = 15000;  
let policyType = "Health";
```

Explain:

A variable is a named container used to store information.

Draw the mental model:

```
applicantName → "Priya"  
age           → 32
```

```
premium      → 15000
policyType    → "Health"
```

7. `let`

Show values changing.

JavaScript

```
let premium = 12000;

console.log(premium);

premium = 15000;

console.log(premium);
```

Insurance explanation:

Premium can change after calculating age, coverage, smoker status, etc.

8. `const`

JavaScript

```
const policyNumber = "POL10001";

console.log(policyNumber);
```

Explain:

Use `const` when the variable should not be reassigned.

Example:

JavaScript

```
const policyType = "Health";
```

Don't overcomplicate `const` vs immutability at this stage.

9. Data Types

Use a single insurance object conceptually, but don't introduce objects yet.

String

JavaScript

```
let customerName = "Anitha";  
let policyType = "Motor";
```

Number

JavaScript

```
let age = 42;  
let premium = 18500;
```

Boolean

JavaScript

```
let policyActive = true;  
let claimApproved = false;
```

Ask students:

What type of data would you use for "Is the policy active?"

Answer:

JavaScript

```
boolean
```

10. Operators

Insurance examples make this very easy.

Arithmetic

JavaScript

```
let basePremium = 10000;  
let tax = 1800;  
  
let totalPremium = basePremium + tax;
```

```
console.log(totalPremium);
```

Other examples:

JavaScript

```
let annualPremium = 24000;  
let monthlyPremium = annualPremium / 12;  
  
console.log(monthlyPremium);
```

11. Comparison Operators

This is one of the **most important sections for testers**.

Use:

JavaScript

```
let expectedPremium = 15000;  
let actualPremium = 15000;  
  
console.log(expectedPremium === actualPremium);
```

Output:

```
true
```

Then:

JavaScript

```
let actualPremium = 16000;  
  
console.log(expectedPremium === actualPremium);
```

Output:

```
false
```

Teach:

```
===    equal  
!==    not equal  
>     greater than
```

<	less than
>=	greater than or equal
<=	less than or equal

12. Expected vs Actual Values

This should be a **major tester exercise**.

JavaScript

```
let expectedPolicyStatus = "Approved";
let actualPolicyStatus = "Approved";

if (expectedPolicyStatus === actualPolicyStatus) {
  console.log("TEST PASS");
} else {
  console.log("TEST FAIL");
}
```

Then intentionally fail it:

JavaScript

```
let expectedPolicyStatus = "Approved";
let actualPolicyStatus = "Rejected";
```

Students should see:

TEST FAIL

Explain:

This is the basic foundation of assertion thinking.

13. Conditional Statements

if

Insurance eligibility:

JavaScript

```
let age = 35;
```



```
if (age >= 18) {  
  console.log("Applicant is eligible");  
}
```

if...else

JavaScript

```
let age = 16;  
  
if (age >= 18) {  
  console.log("Eligible");  
} else {  
  console.log("Not eligible");  
}
```

else if

Introduce different insurance categories:

JavaScript

```
let age = 65;  
  
if (age < 18) {  
  console.log("Not eligible");  
} else if (age <= 60) {  
  console.log("Standard eligibility");  
} else {  
  console.log("Senior eligibility");  
}
```

This is a very good exercise because testers naturally encounter **multiple business rules**.

14. Logical Operators

Use insurance underwriting conditions.

AND — &&

JavaScript

```
let age = 35;  
let income = 600000;
```

```
if (age >= 18 && income >= 300000) {  
  console.log("Eligible for policy");  
}
```

Both conditions must be true.

OR — ||

JavaScript

```
let policyType = "Health";  
let age = 65;  
  
if (policyType === "Health" || age >= 60) {  
  console.log("Special processing required");  
}
```

NOT — !

JavaScript

```
let policyActive = false;  
  
if (!policyActive) {  
  console.log("Policy is inactive");  
}
```

15. Console Output

Teach them to use the console as a **debugging tool**, not just a way to print things.

JavaScript

```
let applicantName = "Kumar";  
let age = 45;  
let premium = 22000;  
  
console.log("Applicant:", applicantName);  
console.log("Age:", age);  
console.log("Premium:", premium);
```

Then:

JavaScript

```
console.log("Starting validation");
```

```
console.log("Expected:", expectedPremium);  
console.log("Actual:", actualPremium);
```

16. Reading Basic Errors

This is particularly important for zero-programming-literacy learners.

Give them deliberately broken code.

Error 1

JavaScript

```
let applicantName = "Ravi"  
  
console.log(applicantName
```

Ask:

What is missing?

Answer:

```
)
```

Error 2

JavaScript

```
console.log(applicantName);
```

without defining `applicantName`.

Explain:

ReferenceError

Error 3

JavaScript

```
let age = 25;  
  
if age >= 18 {  
    console.log("Eligible");  
}
```

```
}
```

Explain syntax error.

Teach them to look at:

1. Error type
2. Error message
3. Line number
4. Character/location where the problem occurred

Day 4 — Tester-Oriented Insurance Hands-on

I would make this a **mini insurance validation exercise** rather than unrelated JavaScript exercises.

Exercise 1 — Policy Status

JavaScript

```
let expectedStatus = "Approved";
let actualStatus = "Approved";

if (expectedStatus === actualStatus) {
  console.log("TEST PASS");
} else {
  console.log("TEST FAIL");
}
```

Exercise 2 — Applicant Age

JavaScript

```
let age = 25;

if (age >= 18) {
  console.log("PASS - Applicant is eligible");
} else {
  console.log("FAIL - Applicant is not eligible");
}
```

Exercise 3 — Premium Range

Requirement:

Premium must be between ₹10,000 and ₹50,000.

JavaScript

```
let premium = 25000;

if (premium >= 10000 && premium <= 50000) {
  console.log("PASS - Premium is within valid range");
} else {
  console.log("FAIL - Premium is outside valid range");
}
```

Exercise 4 — Insurance Application

Give them:

```
Applicant Age = 35
Annual Income = 600000
Premium = 24000
Policy Status = Approved
```

Requirements:

```
Age >= 18
Income >= 300000
Premium between 10000 and 50000
Status = Approved
```

They need to write JavaScript to validate all four conditions.

Day 5 — JavaScript Fundamentals for Testers – II

Day 5 should move from **individual conditions** → **reusable logic** → **test data**.

1. Loops

Start with a simple insurance scenario.

JavaScript

```
for (let i = 1; i <= 5; i++) {
  console.log("Processing policy", i);
}
```

Then explain why loops matter in testing:

Instead of writing the same test logic five times, we can execute it against five pieces of test data.

2. Insurance Test Data

For example:

JavaScript

```
let ages = [18, 25, 35, 45, 60];
```

Then:

JavaScript

```
for (let i = 0; i < ages.length; i++) {  
  console.log("Testing age:", ages[i]);  
}
```

This naturally introduces:

- arrays
 - indexes
 - `.length`
 - loops
-

3. `while`

JavaScript

```
let policyNumber = 1;  
  
while (policyNumber <= 5) {  
  console.log("Processing policy:", policyNumber);  
  policyNumber++;  
}
```

Explain carefully why `policyNumber++` is necessary.

4. Functions

Start extremely simple:

JavaScript

```
function sayHello() {
```

```
    console.log("Welcome to ABC Insurance");  
  }  
  
  sayHello();  
}
```

Then move to useful tester logic:

JavaScript

```
function checkAge(age) {  
  if (age >= 18) {  
    console.log("Eligible");  
  } else {  
    console.log("Not eligible");  
  }  
}  
  
checkAge(25);  
checkAge(16);
```

5. Function Parameters

This is where functions start becoming useful.

JavaScript

```
function validatePremium(premium) {  
  if (premium >= 10000 && premium <= 50000) {  
    return true;  
  }  
  
  return false;  
}
```

Use it:

JavaScript

```
let result = validatePremium(25000);  
  
console.log(result);
```

Then:

JavaScript

```
if (validatePremium(25000)) {  
  console.log("TEST PASS");  
}
```

```
} else {  
    console.log("TEST FAIL");  
}
```

This is an excellent bridge to Playwright assertions later.

6. Return Values

Explain:

JavaScript

```
return true;
```

means:

Give the result back to the place where the function was called.

Insurance example:

JavaScript

```
function isEligible(age) {  
    return age >= 18;  
}  
  
let result = isEligible(35);  
  
console.log(result);
```

7. Strings

Insurance examples:

JavaScript

```
let policyType = "Health Insurance";
```

Teach:

JavaScript

```
policyType.length  
policyType.toUpperCase()  
policyType.toLowerCase()
```



```
policyType.includes("Health")
```

Example:

JavaScript

```
let policyType = "Health Insurance";

if (policyType.includes("Health")) {
  console.log("Health policy selected");
}
```

8. Arrays

Use actual test data:

JavaScript

```
let policyTypes = [
  "Health",
  "Motor",
  "Travel",
  "Life"
];
```

Access values:

JavaScript

```
console.log(policyTypes[0]);
console.log(policyTypes[1]);
```

Explain that array indexes start at 0.

9. Iterating Through Test Data

JavaScript

```
let policyTypes = [
  "Health",
  "Motor",
  "Travel",
  "Life"
];
```

```
for (let i = 0; i < policyTypes.length; i++) {  
  console.log("Testing policy:", policyTypes[i]);  
}
```

Then introduce a realistic test-data table concept:

```
Health  
Motor  
Travel  
Life
```

and explain that later these values might come from:

- Excel
- JSON
- CSV
- API
- database
- test fixtures

This prepares them for automation frameworks.

10. Objects

Now introduce key-value data.

JavaScript

```
let applicant = {  
  name: "Ravi",  
  age: 35,  
  policyType: "Health",  
  premium: 25000  
};
```

Access:

JavaScript

```
console.log(applicant.name);  
console.log(applicant.age);  
console.log(applicant.policyType);  
console.log(applicant.premium);
```

Explain:

```
name      → "Ravi"
```

```
age      → 35
policyType → "Health"
premium  → 25000
```

11. Object-Based Validation

This is an excellent insurance-domain exercise.

JavaScript

```
let applicant = {
  name: "Ravi",
  age: 35,
  income: 600000,
  premium: 25000
};

if (applicant.age >= 18 &&
    applicant.income >= 300000 &&
    applicant.premium >= 10000 &&
    applicant.premium <= 50000) {

  console.log("PASS - Applicant eligible");

} else {

  console.log("FAIL - Applicant not eligible");

}
```

Now students are combining:

variables + objects + operators + conditions + logical operators

12. Basic Error Handling

Introduce only the basics.

JavaScript

```
try {
  let result = 100 / 0;
  console.log(result);
} catch (error) {
  console.log("An error occurred");
}
```

Then explain that error handling becomes much more important when working with:

- APIs
- files
- databases
- Playwright
- external systems

Don't spend too much time on advanced error handling on Day 5.

13. Debugging

Use VS Code.

Give them:

JavaScript

```
let premium = 25000;
let minimumPremium = 10000;
let maximumPremium = 20000;

if (premium >= minimumPremium && premium <= maximumPremium) {
  console.log("PASS");
} else {
  console.log("FAIL");
}
```

Ask:

Why did this produce FAIL?

They should inspect:

```
premium = 25000
minimum = 10000
maximum = 20000
```

Then introduce:

JavaScript

```
console.log(premium);
console.log(minimumPremium);
console.log(maximumPremium);
```

And then VS Code breakpoints.

14. Reading Existing JavaScript

Give them a small insurance function:

JavaScript

```
function validateApplication(age, income, premium) {  
  
    if (age < 18) {  
        return "Rejected";  
    }  
  
    if (income < 300000) {  
        return "Rejected";  
    }  
  
    if (premium < 10000 || premium > 50000) {  
        return "Rejected";  
    }  
  
    return "Approved";  
}  
  
let result = validateApplication(35, 600000, 25000);  
  
console.log(result);
```

Ask students to answer:

1. What are the parameters?
2. What values are passed?
3. What does the function return?
4. When does it return `Rejected`?
5. When does it return `Approved`?

This teaches them to **read automation code**, not merely write it.

15. Breaking Problems into Small Logical Steps

This should be the final concept of Day 5.

Give them this requirement:

A motor insurance customer can proceed if they are at least 18 years old, have a valid driving licence, and the vehicle age is not more than 15 years.

Instead of asking them to immediately write code, teach them to break it down:

Step 1 — Identify inputs

```
age
hasDrivingLicence
vehicleAge
```

Step 2 — Identify rules

```
age >= 18
hasDrivingLicence = true
vehicleAge <= 15
```

Step 3 — Combine rules

```
age >= 18
AND
hasDrivingLicence
AND
vehicleAge <= 15
```

Step 4 — Write JavaScript

JavaScript

```
let age = 35;
let hasDrivingLicence = true;
let vehicleAge = 5;

if (age >= 18 &&
    hasDrivingLicence === true &&
    vehicleAge <= 15) {

    console.log("PASS - Eligible for motor insurance");

} else {

    console.log("FAIL - Not eligible");

}
```

That **problem** → **inputs** → **rules** → **conditions** → **code** → **result** process is probably more valuable for your learners than teaching JavaScript syntax in isolation.

Suggested Day 4/5 Insurance Mini Project

At the end of Day 5, give them a small Insurance Policy Eligibility Validator.

Input

JavaScript

```
let applicant = {  
  name: "Anitha",  
  age: 35,  
  income: 750000,  
  policyType: "Health",  
  premium: 30000,  
  policyActive: true  
};
```

Business Rules

1. Age must be ≥ 18
2. Income must be ≥ 300000
3. Premium must be between 10000 and 50000
4. Policy type must be Health, Motor, Life or Travel
5. Policy must be active

Expected output

Applicant: Anitha
Policy Type: Health
Eligibility: PASS

Then give them deliberately invalid data:

JavaScript

```
let applicant = {  
  name: "Anitha",  
  age: 16,  
  income: 200000,  
  policyType: "Health",  
  premium: 70000,  
  policyActive: false  
};
```

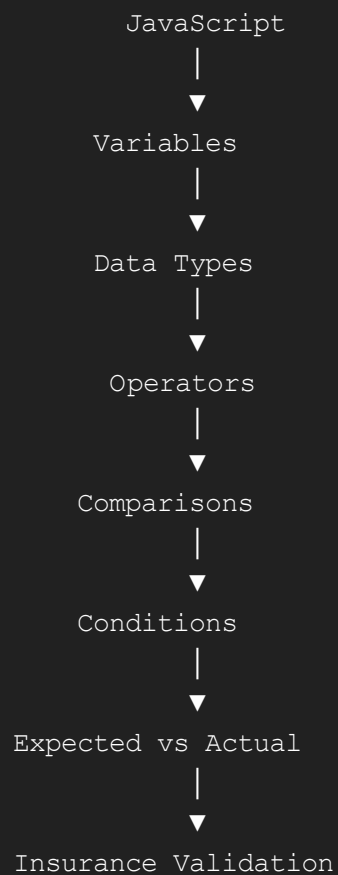
Expected:

Eligibility: FAIL

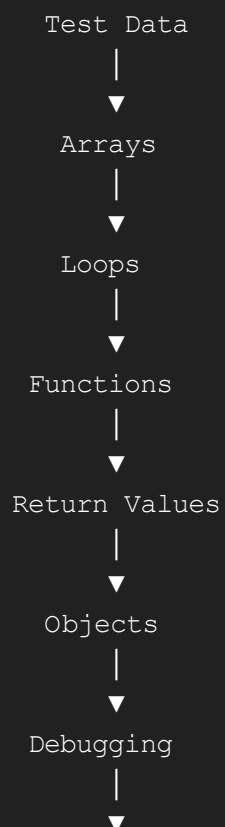
The learning progression

I would structure the two days around this progression:

DAY 4



DAY 5



Insurance Validator



Playwright Readiness

The key is **not to teach JavaScript as a developer course**. Teach it as **the programming language needed to express a tester's thinking**:

Requirement → Test Data → Condition → Validation → Pass/Fail

That will make the eventual transition to **Playwright + TypeScript** considerably smoother.



 Jotform

Ad

Replit App Maker Alternative

A simpler Replit app maker alternative—build apps in ChatGPT with Jotform, no code.